

From GUI products to Claude APIs

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 4 of 12 · English notes

After the product recap, session 4 treated *APIs* in two layers that Foundation candidates mix up: (1) the **OpenAPI contract** of the app you are generating (REST, Swagger UI, design-first), and (2) the fact that Claude Code itself is a client that will later be rebuilt with **Claude APIs / the Agent SDK**. This chapter is the first layer, plus how GUI clients (Code, Cowork Design) produce that contract and a runnable backend without you typing handlers.

DOCS MOVE; CONTRACTS STAY

OpenAPI 3.1, Swagger UI paths, and Claude Code skill folder names in the demo may already look different. The exam cares that you can explain design-first APIs, how a skill injects an API architect role, and that attaching a YAML is cheaper than attaching a whole backend folder. Confirm current Claude API docs at [the API overview](#) when you reach Modules 07+.

Design-first OpenAPI, not “code then document”

The trainer framed the first artefact a full-stack team should freeze as an **OpenAPI specification** (he mentioned W3C/CNCF in passing; the practical standard in the demo is OpenAPI, e.g. 3.1, with Swagger-style docs). The method is **design-first**: write the contract, circulate it to backend and frontend, then implement. That is the opposite of generating code and hoping the docs catch up.

- Role: API architect / designer — not the same skill as “principal solution architect,” though both consume the SRS.
- Contents: paths, request/response models, auth (the demo later needed a JWT secret in `.env`), GET vs POST vs PUT as the architect decides.
- Output in the lab: `openapi.yaml` in the repo, then backends generated *from* that file rather than from a vague “build me an API” prompt.
- Frontend later uses the same YAML (or generated types) so screens and fields do not invent columns the API does not have.

An OpenAPI skill is a slash-invoked role

Like the analyst and architect skills, the OpenAPI skill lives as `SKILL.md` under a named folder (the trainer created something he referred to as an “open API” slash). The body says: act as API architect, follow OpenAPI 3.1, produce Swagger documentation, request and response models, microservice-shaped resources if that is the architecture. HTTP verbs are not trivia: if the architect wants PATCH instead of PUT, that belongs in the spec and in the skill, not as a surprise in generated Express/FastAPI code.

You attach the existing BRD/SRS (or HLD) into the prompt, invoke the slash, and ask for YAML in a known folder (e.g. docs/). Refresh the skill list if the new slash does not appear. The YAML then becomes the attachment of choice for the next skills — cheaper than re-attaching the whole backend/ tree.

Backend from the contract: FastAPI, Node, or whatever the memory says

A **backend skill** plus a backend-scoped CLAUDE.md tell Claude Code which runtime to use (the live demo used Node-style package.json, npm run build / npm start, port 3000). The instruction “implement the APIs only, from this OpenAPI file” keeps the agent from inventing a parallel schema. Nested memory matters: the orchestrator can load global CLAUDE.md plus backend/CLAUDE.md for that sub-task (Module 05 expands the agent split).

1. New session for a new activity so context does not blend architect docs with implementation noise.
2. Slash the backend skill; attach openapi.yaml; name the output folder.
3. Let the five-phase loop run: clarify, plan (folders, controllers, services), execute, verify (tests — 43 passing in the demo), deliver a summary.
4. Run it yourself or ask Claude to run it. Missing .env (JWT secret) is a typical first failure; that is configuration, not a model bug.
5. If Swagger UI was omitted from the skill, a second pass (“include Swagger UI”) works but roughly *doubles* tokens because the model must re-read the tree. Put it in CLAUDE.md or the skill the first time.

VERIFICATION TAX

Claude Code may try to prove clean-code compliance and re-run tests on every file. That is the “verify” phase. Disable or defer it in memory if you will smoke-test locally. The trainer still used tests as a quality bar once Swagger was added — 43 cases passing — so “skip verify” is a cost control, not a claim that tests do not matter.

What “Claude APIs” means for Foundation from this point

Two sentences from the session that you should not collapse into one: *we will implement MCP tools of our own when we implement APIs*, and *Claude Code is already an agentic client on top of Claude APIs*. Session 4 does not install the Python/TS SDK. It shows that the GUI products are not a dead end: they already call models with tools, store tool results in a sandbox, and can generate REST services you then hit from a browser. Modules 07+ make the Messages API explicit. Here, the exam-relevant move is: GUI client → artefacts (OpenAPI, code) → a real HTTP API you can curl or open in Swagger.

Classroom proof that the backend is real

After `npm run build` and `npm start`, Swagger UI listed `register / login / logout / refresh` and lock-marked routes. Try-it-out created a user, login returned a bearer token, `authorized /me` (or equivalent) returned the user. No application code was typed. Failures (wrong host, missing secret, Swagger not in first generation) were ordinary engineering, not “the LLM refused.”

Cowork Design as another GUI path into the same APIs

To seed a frontend, the trainer switched to **Cowork** → Design (lower left in the then-current UI) and asked for wireframes of a task-management app. He attached the OpenAPI YAML rather than the whole backend folder specifically to save tokens. A branding/logo file would have been the other high-value attachment. Clarify-phase questions appeared (desktop vs responsive, personal vs company-wide, Kanban vs list) — answer narrowly if you are token-budgeting (desktop + personal in the demo).

A learner noted that industry often designs UI first, then schema. The trainer’s lab order is the reverse on purpose: the YAML already lists the fields; Design reads schemas and proposes screens (login, register, dashboard, CRUD). You can still import HTML into Figma (HTML-to-design plugins) if UX will polish there. Editing in Cowork Design is “a little expensive”; Figma is for humans who will iterate visually. That Figma file becomes the MCP input in the next chapter.

Exam posture

Foundation is not “write FastAPI by hand.” It is: know that design-first OpenAPI is the shared language between agents, that skills and CLAUDE.md steer generation, that a second generation pass is a token penalty, and that Chat/Cowork/Code are how you reach models until you call the Messages API yourself. Keep product names in English in your written answers.

Official documentation

- **API overview**

<https://docs.claude.com/en/api/overview>

Where GUI clients are heading: programmatic Claude, not just Chat.

- **Messages API**

<https://docs.claude.com/en/api/messages>

The request shape later modules implement; Code already wraps this.

- **Claude Code overview**

<https://code.claude.com/docs/en/overview>

Skills, memory, and running generated services from the agent.

- **Workspaces**

<https://docs.claude.com/en/docs/build-with-claude/workspaces>

Org-level model access that limits which APIs and models you can call.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.